

```
from mcmc import *

def MetropolisHastingsGibbs(q0, s2_0, rod, x_obs, T_obs, D, M, ns, sigma2_s, rng_seed=None):
    """
    This function alternates between:
    1. Metropolis-Hastings step for q (O|, h) given current sigma_squared
    2. Gibbs step for sigma_squared given current q

    Parameters:
    q0: initial q sample [Phi, h]
    s2_0: initial sigma_squared value
    rod: heat rod object for model evaluation
    x_obs: observation locations
    T_obs: observed temperatures
    D: proposal covariance matrix for q
    M: number of samples to generate
    ns, sigma2_s: hyperparameters for inverse gamma prior on sigma_squared
    rng_seed: random seed for reproducibility

    Returns:
    q_samples: array of q samples (M x 2)
    s2_samples: array of sigma_squared samples (M)
    acceptance_rate: acceptance rate of the chain for q
    """
    if rng_seed is not None:
        np.random.seed(rng_seed)

    q_current = np.array(q0, dtype=float)
    s2_current = s2_0
    n_obs = len(x_obs)

    q_samples = np.zeros((M, len(q0)))
    s2_samples = np.zeros(M)

    q_samples[0] = q_current
    s2_samples[0] = s2_current

    # Prior parameters
    prior_Phi_mean = -15
    prior_Phi_std = 10
    prior_h_mean = 0.001
    prior_h_std = 0.005

    accepted = 0

    print("Running Metropolis-Hastings with Gibbs sampling for unknown sigma_squared...")
    print(f"ns = {ns}, sigma2_s = {sigma2_s}")
    print(f"Initial sigma_squared = {s2_0}")

    for k in range(1, M):
        # =====
        # STEP 2.1: METROPOLIS-HASTINGS FOR q GIVEN CURRENT sigma^2
        # =====

        # proposal from N(q_current, D)
        q_proposed = np.random.multivariate_normal(q_current, D)

        # Compute acceptance ratio r(q*, q_{k-1}) using current sigma_squared
        # r = [pi(data|q*,sigma_squared)pi0(q*)] / [pi(data|q_{k-1},sigma_squared)pi0(q_{k-1})] (symmetric J(q*|q_{k-1}))

        # Evaluate likelihood for proposed q
        Phi_star, h_star = q_proposed
        rod.Phi_val = Phi_star
        rod.h_val = h_star
        T_model_star = rod.Ts(x_obs)
        log_likelihood_star = -0.5 * np.sum((T_obs - T_model_star)**2 / s2_current)

        # Evaluate likelihood for current q
        Phi_curr, h_curr = q_current
        rod.Phi_val = Phi_curr
        rod.h_val = h_curr
        T_model_curr = rod.Ts(x_obs)
        log_likelihood_curr = -0.5 * np.sum((T_obs - T_model_curr)**2 / s2_current)

        # Log priors for q
        log_prior_star = (norm.logpdf(Phi_star, prior_Phi_mean, prior_Phi_std) +
                          norm.logpdf(h_star, prior_h_mean, prior_h_std))
        log_prior_curr = (norm.logpdf(Phi_curr, prior_Phi_mean, prior_Phi_std) +
                          norm.logpdf(h_curr, prior_h_mean, prior_h_std))

        # Log ratio (proposal is symmetric, so it cancels out)
        log_ratio = (log_likelihood_star + log_prior_star) - (log_likelihood_curr + log_prior_curr)
        r = np.exp(log_ratio)

        # Accept or reject
        if np.random.random() < min(1, r):
            q_current = q_proposed
            accepted += 1

        q_samples[k] = q_current

        # =====
        # STEP 2.2: GIBBS STEP FOR sigma_squared GIVEN CURRENT q
        # =====

        # Compute sum of squared errors for current q
        Phi, h = q_current
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)
        SSq = np.sum((T_obs - T_model)**2)

        # Inverse Gamma parameters for Gibbs sampling
        # From textbook p. 163: p(sigma_squared|data,q) = Inv-gamma(O± + n/2, OI + SSq/2)
        # where O± = ns/2, OI = (ns * sigma_squared_s)/2
        aval = ns/2 + n_obs/2
        bval = (ns * sigma2_s)/2 + SSq/2

        # Sample from Inv-Gamma(aval, bval)
        # Using the relationship:
        # If X ~ Gamma(shape=aval, rate=bval), then 1/X ~ Inv-Gamma(aval, bval)
        # numpy.random.gamma requires shape and scale (1/rate)
        gamma_sample = np.random.gamma(aval, 1/bval)
        s2_current = 1 / gamma_sample

        s2_samples[k] = s2_current

    # Progress reporting
    if (k + 1) % 1000 == 0:
        print(f" Completed {k + 1}/{M} iterations, MCMC acceptance rate: {accepted/(k):.3f}, sigma_squared = {s2_current:.4f}")

    acceptance_rate = accepted / (M - 1)
```

```
    return q_samples, s2_samples, acceptance_rate

def run_metropolis_gibbs_for_heat_equation(M=10000,D = None, data_name=None, ns=0.01, sigma2_s=4.0, burnin=300):
    """
    Run Metropolis-Hastings with Gibbs sampling for unknown sigma_squared

    Parameters:
    M: number of samples (default 10000)
    data_name: MATLAB data file name
    ns, sigma2_s: hyperparameters for inverse gamma prior on sigma_squared
    burnin: number of burn-in samples to remove
    """
    if data_name is None:
        data_name = 'HW06_Problem3.mat'

    rod, x_obs, T_obs, sigma2_0, q0 = setup_heat_equation_problem(data_name)

    # Proposal covariance matrix for q
    if D is None:
        D = np.array([[1e-2, 0],
                       [0, 2e-10]])

    # Initial sigma_squared (use the known value from problem 1.1 as starting point)
    s2_0 = sigma2_0

    print("\n" + "="*70)
    print("PROBLEM 1.2: METROPOLIS-HASTINGS WITH GIBBS FOR UNKNOWN sigma_squared")
    print("="*70)
    print(f"Hyperparameters: ns = {ns}, sigma_squared_s = {sigma2_s}")
    print(f"Initial sigma_squared = {s2_0}")
    print(f"Running for M = {M} iterations with {burnin} burn-in samples")

    # MCMC with Gibbs
    q_samples, s2_samples, acceptance_rate = MetropolisHastingsGibbs(
        q0, s2_0, rod, x_obs, T_obs, D, M, ns, sigma2_s, rng_seed=42
    )

    print(f"\nFinal acceptance rate for q: {acceptance_rate:.3f}")

    # Remove burn-in samples
    q_samples_post = q_samples[burnin:]
    s2_samples_post = s2_samples[burnin:]

    # Prior parameters for q (for MAP calculation)
    prior_Phi_mean = -15
    prior_Phi_std = 10
    prior_h_mean = 0.001
    prior_h_std = 0.005

    # Find MAP estimate (maximum unnormalized posterior with sigma_squared)
    print("\nComputing posterior values for MAP estimate...")

    posterior_values = np.zeros(len(q_samples_post))
    for i, (q, s2) in enumerate(zip(q_samples_post, s2_samples_post)):
        Phi, h = q
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)

        # Likelihood with current sigma_squared
        n = len(x_obs)
        likelihood = (1/np.sqrt(2*np.pi*s2))**n * np.exp(-0.5 * np.sum((T_obs - T_model)**2 / s2))

        # Prior for q
        prior_q = (norm.pdf(Phi, prior_Phi_mean, prior_Phi_std) *
                   norm.pdf(h, prior_h_mean, prior_h_std))

        # Prior for sigma_squared (Inverse Gamma)
        # p(sigma_squared) \propto (sigma_squared)^(-(ns/2+1)) * exp(-(ns*sigma_squared_s)/(2sigma_squared))
        log_prior_sigma = -(ns/2 + 1) * np.log(s2) - (ns * sigma2_s) / (2 * s2)
        prior_sigma = np.exp(log_prior_sigma)

        # Unnormalized posterior
        posterior_values[i] = likelihood * prior_q * prior_sigma

    map_idx = np.argmax(posterior_values)
    q_map = q_samples_post[map_idx]
    s2_map = s2_samples_post[map_idx]
    post_map = posterior_values[map_idx]

    print(f"\n{' '*60}")
    print("PROBLEM 1.2 RESULTS (with unknown sigma_squared)")
    print(f"{' '*60}")
    print(f"nMAP Estimate (q_MAP, sigma_squared_MAP):")
    print(f"  O| = {q_map[0]:.6f} W/cmBI")
    print(f"  h = {q_map[1]:.6f} W/cmBI/B°C")
    print(f"  sigma_squared = {s2_map:.6f} (B°C)BI")
    print(f"  sigma = {np.sqrt(s2_map):.4f} B°C")
    print(f"  pi(data|q_MAP,sigma_squared_MAP)pi0(q_MAP,sigma_squared_MAP) = {post_map:.2e}")

    print(f"\nPosterior Distribution pi(q,sigma_squared|data) Summary (after burn-in):")
    print(f"{' '*60}")
    print(f"O|:")
    print(f"  Mean = {np.mean(q_samples_post[:, 0]):.6f} W/cmBI")
    print(f"  Std  = {np.std(q_samples_post[:, 0]):.6f} W/cmBI")
    print(f"  95% CI = [{np.percentile(q_samples_post[:, 0], 2.5):.6f}, "
          f"{np.percentile(q_samples_post[:, 0], 97.5):.6f}]"")

    print(f"nh:")
    print(f"  Mean = {np.mean(q_samples_post[:, 1]):.6e} W/cmBI/B°C")
    print(f"  Std  = {np.std(q_samples_post[:, 1]):.6e} W/cmBI/B°C")
    print(f"  95% CI = [{np.percentile(q_samples_post[:, 1], 2.5):.6e}, "
          f"{np.percentile(q_samples_post[:, 1], 97.5):.6e}]"")

    print(f"nsigma_squared:")
    print(f"  Mean = {np.mean(s2_samples_post):.6f} (B°C)BI")
    print(f"  Std  = {np.std(s2_samples_post):.6f} (B°C)BI")
    print(f"  95% CI = [{np.percentile(s2_samples_post, 2.5):.6f}, "
          f"{np.percentile(s2_samples_post, 97.5):.6f}]"")

    # Create diagnostic plots
    fig, axes = plt.subplots(2, 3, figsize=(18, 12))

    # Plot 1: Phi histogram
    axes[0, 0].hist(q_samples_post[:, 0], bins=30, density=True, alpha=0.7,
                   color='skyblue', edgecolor='black')
    axes[0, 0].axvline(q_map[0], color='red', linestyle='--', linewidth=2,
                      label=f'MAP: {q_map[0]:.4f}')
    axes[0, 0].set_xlabel('O| (W/cmBI)')
    axes[0, 0].set_ylabel('Density')
    axes[0, 0].set_title('Marginal Posterior pi(O|data)')
    axes[0, 0].legend()
```

```

axes[0, 0].grid(True, alpha=0.3)

# Plot 2: h histogram
axes[0, 1].hist(q_samples_post[:, 1], bins=30, density=True, alpha=0.7,
               color='lightcoral', edgecolor='black')
axes[0, 1].axvline(q_map[1], color='red', linestyle='--', linewidth=2,
                  label=f'MAP: {q_map[1]:.6f}')
axes[0, 1].set_xlabel('h (W/cmBI/B°C)')
axes[0, 1].set_ylabel('Density')
axes[0, 1].set_title('Marginal Posterior pi(h|data)')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Plot 3: sigma_squared histogram with KDE
axes[0, 2].hist(s2_samples_post, bins=30, density=True, alpha=0.7,
               color='lightgreen', edgecolor='black', label='Histogram')

# Add KDE
from scipy.stats import gaussian_kde
kde = gaussian_kde(s2_samples_post)
x_range = np.linspace(s2_samples_post.min(), s2_samples_post.max(), 200)
axes[0, 2].plot(x_range, kde(x_range), 'b-', linewidth=2, label='KDE')
axes[0, 2].axvline(s2_map, color='red', linestyle='--', linewidth=2,
                  label=f'MAP: {s2_map:.4f}')
axes[0, 2].set_xlabel('sigma_squared (B°C)BI')
axes[0, 2].set_ylabel('Density')
axes[0, 2].set_title('Marginal Posterior pi(sigma_squared|data)')
axes[0, 2].legend()
axes[0, 2].grid(True, alpha=0.3)

# Plot 4: Joint posterior scatter (Phi vs h) colored by sigma_squared
scatter = axes[1, 0].scatter(q_samples_post[:, 0], q_samples_post[:, 1],
                             alpha=0.5, c=s2_samples_post, cmap='viridis', s=10)
axes[1, 0].scatter(q_map[0], q_map[1], color='red', s=200, marker='*', label='MAP')
axes[1, 0].set_xlabel('O (W/cmBI)')
axes[1, 0].set_ylabel('h (W/cmBI/B°C)')
axes[1, 0].set_title('Joint Posterior Samples (colored by sigma_squared)')
plt.colorbar(scatter, ax=axes[1, 0], label='sigma_squared (B°C)BI')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Plot 5: Model fit with MAP parameters
rod.Phi_val = q_map[0]
rod.h_val = q_map[1]
T_map = rod.Ts(x_obs)

axes[1, 1].plot(x_obs, T_obs, 'ko', markersize=8, label='Observations')
axes[1, 1].plot(x_obs, T_map, 'r-', linewidth=2, label='MAP prediction')
axes[1, 1].fill_between(x_obs, T_map - 2*np.sqrt(s2_map), T_map + 2*np.sqrt(s2_map),
                       color='red', alpha=0.2, label=f'B±2Πf̂ (Πf̂={np.sqrt(s2_map):.2f}B°C)')
axes[1, 1].set_xlabel('Position x (cm)')
axes[1, 1].set_ylabel('Temperature (B°C)')
axes[1, 1].set_title('Model Fit with MAP Parameters')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

# Plot 6: Trace plot of sigma_squared
axes[1, 2].plot(range(M), s2_samples, 'g-', alpha=0.5, linewidth=0.5)
axes[1, 2].axhline(s2_map, color='red', linestyle='--', label=f'MAP: {s2_map:.4f}')
axes[1, 2].axvline(burnin, color='black', linestyle=':', label=f'Burn-in ({burnin})')
axes[1, 2].set_xlabel('Iteration')
axes[1, 2].set_ylabel('sigma_squared (B°C)BI')
axes[1, 2].set_title('Trace Plot of sigma_squared Samples')
axes[1, 2].legend()
axes[1, 2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('problem1.2_results.png', dpi=150)
plt.show()

return q_samples, s2_samples, q_map, s2_map, posterior_values, acceptance_rate

```